

Lock-Based and Time Stamp Protocols in Database Management Systems

Summary

Concurrency control mechanisms in **Database Management Systems (DBMS)** are designed to ensure **serializability, atomicity, and consistency** when multiple transactions execute simultaneously. This briefing analyzes two major families of protocols:

- **Lock-Based Protocols**, centered around **Two-Phase Locking (2PL)** and its variants.
- **Time Stamp Ordering Protocols (TSP)**, including the **Thomas Write Rule (TWR)**.

The discussion highlights how these protocols prevent anomalies, ensure recoverability, and manage trade-offs involving **deadlock, starvation, rollback cost, and protocol rigidity**—all of which are frequent examination themes.

1. Lock-Based Protocols

Lock-based protocols enforce mutual exclusion by requiring transactions to obtain locks before accessing data items.

1.1 Fundamental Lock Types

- **Shared Lock (S)**
 - Used for **Read** operations
 - Multiple transactions may hold an S-lock on the same data item simultaneously
- **Exclusive Lock (X)**
 - Required for **Write** (or Read–Write) operations
 - Only one transaction may hold an X-lock on a data item
 - Blocks both S and X locks from other transactions

1.2 Two-Phase Locking (2PL)

Two-Phase Locking divides transaction execution into two strictly ordered phases:

1. Growing Phase

- Locks may be acquired
- No locks may be released

2. Shrinking Phase

- Locks may be released
- No new locks may be acquired

🔑 Key Property:

All 2PL-based protocols **guarantee conflict serializability**.

1.3 Variations of 2PL

Protocol	Locking Rule	Serializability	Recoverability	Deadlock / Starvation
Basic 2PL	Growing + Shrinking phases	Yes	No	Suffers both
Strict 2PL	Hold all X-locks till commit	Yes	Yes	Suffers both
Rigorous 2PL	Hold all locks (S + X) till commit	Yes	Yes	No deadlock, starvation possible
Conservative 2PL	Acquire all required locks at start	Yes	Yes	No deadlock, starvation possible

Exam Insight

- **Strict 2PL** $\Rightarrow$ **Strict schedules** $\Rightarrow$ **Cascadeless** $\Rightarrow$ **Recoverable**
- **Conservative 2PL** eliminates deadlock *by design* but reduces concurrency.

2. Problems in Concurrent Execution

Even with locking, concurrency introduces **system-level risks**.

2.1 Deadlock

Deadlock occurs when transactions form a **cyclic wait condition**.

Example

- T_1 waits for a resource held by T_2
- T_2 waits for a resource held by T_3
- T_3 waits for a resource held by T_1

➡ No transaction can proceed → **infinite wait**

Key Point

- Deadlock = *system-wide halt*
- Requires detection, prevention, or avoidance mechanisms

2.2 Starvation

Starvation occurs when a transaction is **indefinitely postponed** due to repeated preemption by higher-priority transactions.

Characteristics

- System continues executing transactions
- One (or more) low-priority transactions never progress

Contrast with Deadlock

- Deadlock: No transaction progresses
- Starvation: Some transactions progress, others never do

3. Time Stamp Ordering Protocol (TSP)

Time Stamp Ordering is a **non-locking** concurrency control mechanism.

3.1 Mechanism

- Each transaction is assigned a **unique timestamp (TS)** at entry
- Serial order is determined by **increasing timestamp**
- Older transactions must appear before newer ones

Types of Timestamp Usage

- **Transaction Timestamp:** Fixed per transaction
- **Data Item Timestamps:**
 - **RTS(X)** → Highest TS of any transaction that read X
 - **WTS(X)** → Highest TS of any transaction that wrote X

3.2 Protocol Rules and Conflict Detection

TSP enforces **monotonic forward movement** in time.

Allowed

- Older → Newer operations
- Read–Read (RR) in any order (no conflict)

Violations (Cause Rollback)

- Write after a newer Read (WR)
- Read after a newer Write (RW)
- Write after a newer Write (WW)

Backward movement in time = violation

Key Observation

TSP guarantees **conflict serializability and recoverability**, but may cause **frequent rollbacks**.

4. Thomas Write Rule (TWR)

The **Thomas Write Rule** is a refinement of TSP to improve concurrency.

Core Idea: Obsolete Writes

In standard TSP:

- If T_i (older) attempts to write after T_j (newer) has written → rollback T_i

In **TWR**:

- If the write is **obsolete**, it is simply **ignored**
- Transaction is *not rolled back*

What TWR Allows

- Certain **Write–Write (WW)** conflicts

What TWR Still Rejects

- Read–Write (RW)
- Write–Read (WR)

Important Distinction

- TWR increases concurrency
- But may allow **non-view-serializable schedules**

5. Summary of Protocol Compliance

Condition	Basic 2PL	Strict 2PL	TSP	TWR
Ensures Serializability	Yes	Yes	Yes	Yes
Ensures Recoverability	No	Yes	Yes	Yes
Eliminates Deadlock	No	No	Depends	Depends
Handles Obsolete Writes	N/A	N/A	No	Yes